

مهندسی اینترنت

پروژه پایانی

نیم سال دوم سال تحصیلی 1404-1405

مقدمه

در این سند، سه پروژه برای درس مهندسی اینترنت معرفی شده است. هر پروژه یک سامانه تحت وب با سناریوی کاری مشخص را توصیف می‌کند. هدف از این پروژه‌ها این است که دانشجویان بتوانند مفاهیم اصلی درس را در قالب یک سیستم واقعی، قابل اجرا و قابل نمایش پیاده‌سازی کنند. پروژه‌ها به گونه‌ای طراحی شده‌اند که صرفاً مجموعه‌ای از چند صفحه یا چند API جدا از هم نباشند، بلکه یک سامانه منسجم با جریان کاری مشخص، نقش‌های کاربری، ذخیره‌سازی داده، کنترل دسترسی و رابط کاربری قابل استفاده باشند. انتظار می‌رود هر گروه بتواند در زمان ارائه، سناریوی اصلی پروژه را از ابتدا تا انتها اجرا کرده و عملکرد بخش‌های مختلف سامانه را به درستی نشان دهد. پروژه‌ها به صورت گروهی انجام می‌شوند و هر گروه می‌تواند حداکثر دو نفره باشد. هر گروه فقط یکی از پروژه‌های تعریف شده را انتخاب کرده و مسئولیت پیاده‌سازی کامل همان پروژه را بر عهده دارد.

چارچوب فنی پروژه

برای اینکه پروژه‌ها از نظر سطح فنی قابل مقایسه باشند، چارچوب کلی پیاده‌سازی به صورت زیر تعیین می‌شود:

Backend

بخش Backend باید با Node.js پیاده‌سازی شود. استفاده از Express.js برای تعریف Route ها، Middleware ها و API ها توصیه و مورد انتظار است. بخش Backend باید مسئول موارد دریافت درخواست‌های کاربر از طریق API، اعتبارسنجی داده‌های ورودی، اجرای منطق اصلی سامانه و ارسال پاسخ مناسب به Frontend باشد.

Frontend

بخش Frontend باید با استفاده از HTML، CSS و JS خام یا فریم‌ورک‌ها پیاده‌سازی شود. ارتباط Frontend با Backend باید منحصراً از طریق API و ترجیحاً با استفاده از Fetch API انجام شود. استفاده از کتابخانه‌ها و فریم‌ورک‌های Frontend مانند Bootstrap، Tailwind، React، Vue یا ابزارهای مشابه آزاد است؛ اما استفاده از آن‌ها الزامی نیست. آنچه اهمیت دارد، خروجی نهایی، درستی عملکرد، خوانایی رابط کاربری و تجربه کاربری مناسب است، نه صرفاً استفاده از فریم‌ورک‌های پیچیده.

Database

برای ذخیره‌سازی داده‌ها باید از پایگاه داده استفاده شود. پایگاه داده الزامی و پیش‌نهادی برای این پروژه‌ها **MongoDB** است. در صورت استفاده از پایگاه داده دیگر، ساختار داده‌ها، روابط و نحوه اجرای پروژه باید به صورت شفاف توسط گروه توضیح داده شود.

انتظار کلی از پروژه

پروژه انتخاب‌شده باید یک سامانه کامل تحت وب باشد. حداقل انتظار از هر پروژه این است که:

- سناریوی اصلی پروژه به صورت کامل پیاده‌سازی شده باشد.
 - کاربران بتوانند به درستی ثبت‌نام و ورود انجام دهند.
 - نقش‌های کاربری تعریف‌شده به دقت پیاده‌سازی و محدود شده باشند.
 - کنترل دسترسی به صورت امن در سمت سرور انجام شود.
 - داده‌ها در پایگاه داده ذخیره و مدیریت شوند.
 - Frontend و Backend به صورت اصولی از طریق API با یکدیگر ارتباط داشته باشند.
 - کدها خوانا، مرتب و تا حد ممکن ماژولار نوشته شده باشند.
 - پروژه قابل اجرا بوده و در زمان ارائه بدون مشکل سناریوی اصلی را نمایش دهد.
- تمرکز ارزیابی بیشتر بر درستی عملکرد، کامل بودن جریان کاری، هماهنگی اجزای سیستم، رعایت نکات امنیتی پایه و کیفیت پیاده‌سازی است. حجم زیاد کد یا ظاهر بسیار پیچیده، جایگزین منطق درست و پیاده‌سازی کامل نخواهد بود.

طراحی رابط کاربری (UI/UX)

رابط کاربری بخش مهمی از پروژه است و در نمره نهایی تأثیر مستقیم دارد. نیازی به ظاهری بسیار پیچیده یا پرزرق‌وبرق نیست، اما طراحی باید برای راحتی کاربر و بر اساس استانداردهای اولیه صورت گیرد. انتظار می‌رود:

- صفحات اصلی سامانه واضح و قابل فهم باشند.
- مسیرهای اصلی سیستم برای کاربر مشخص و در دسترس باشند.
- فرم‌ها دارای اعتبارسنجی اولیه (Frontend Validation) باشند.
- پیام‌های خطا و موفقیت به شکل مناسب به کاربر نمایش داده شوند.
- وضعیت‌های مختلف با رنگ، برجستگی یا آیکون‌های مناسب متمایز شوند.
- داشبوردها و پنل‌ها اطلاعات مهم را به صورت ساختاریافته و منظم نمایش دهند.
- طراحی کلی پروژه از نظر بصری یکپارچه و منسجم باشد.

نکات اجرایی مهم

- پروژه باید بدون نیاز به تنظیمات پیچیده، قابل اجرا باشد.
- سناریوهای اصلی پروژه باید بدون خطای منطقی یا سیستمی کار کنند.
- اطلاعات حساس مانند رمز عبور نباید به صورت خام (Plain Text) در دیتابیس ذخیره شوند.
- تمامی داده‌های ورودی از سمت کاربر، باید در سمت سرور اعتبارسنجی شوند.
- استفاده از کتابخانه‌ها و ابزارهای کمکی آزاد است، اما منطق اصلی پروژه باید توسط خود گروه پیاده‌سازی شده باشد.

- پروژه‌ای که صرفاً ظاهر داشته باشد ولی منطق اصلی (Backend) آن ناقص باشد، نمره کامل دریافت نخواهد کرد.
- پروژه‌ای که API کامل داشته باشد ولی رابط کاربری قابل استفاده‌ای برای آن طراحی نشده باشد نیز ناقص محسوب می‌شود.
- استفاده از هوش مصنوعی در پیاده سازی **بلامانع** است، اما گروه باید به تمامی فلوها و کدهای نوشته شده تسلط داشته باشد و در صورت نیاز امکان live-coding بدون استفاده از هوش مصنوعی را دارا باشد.

تحويل پروژه

در زمان تحويل، هر گروه باید موارد زیر را ارائه دهد:

- سورس کد کامل هر دو بخش Frontend و Backend
- مستند کوتاه درباره نحوه اجرای پروژه و نصب پیش‌نیازها
- مستند کوتاه از API‌های پیاده‌سازی شده
- آمادگی برای نمایش یک سناریوی کامل از ابتدا تا انتها در جلسه ارزیابی

دانشگاه صنعتی خواجه نصیرالدین طوسی

پروژه ۱: سامانه تمرین و ارزیابی خودکار (KNJudge) HTML/CSS/JS

۱. معرفی سامانه

کیومرث که به تازگی درس مهندسی اینترنت را با موفقیت گذرانده، تصمیم گرفته ترم بعدی بعنوان دستیار این درس فعالیت کند. اما او تصمیم گرفته با ایجاد یک سامانه تمرین و ارزیابی خودکار HTML/CSS/JS تیم حل تمرین و همچنین دانشجویان درس را از بند ارائه های طولانی رها کند! KNJudge یک سامانه تحت وب برای تعریف، ارسال و ارزیابی تمرین های کوچک برنامه نویسی Frontend است. در این سامانه، استاد یا دستیار آموزشی می تواند تمرین هایی مرتبط با HTML، CSS و JavaScript تعریف کند. دانشجویان کد خود را در محیط سامانه وارد کرده، خروجی آن را به صورت زنده مشاهده می کنند و در نهایت نتیجه چند تست ساده روی پاسخ خود را می بینند. هدف اصلی این سامانه، ایجاد یک محیط آموزشی تعاملی برای تمرین مفاهیم پایه وب است. دانشجو می تواند تمرین هایی مانند ساخت فرم، جدول، دکمه، تغییر محتوا با JavaScript، استفاده از classها، انتخاب عناصر DOM و مدیریت رویدادها را انجام دهد. سامانه نیز بر اساس تست های از پیش تعریف شده، پاسخ دانشجو را در مرورگر بررسی می کند.

۲. نقش ها و سطوح دسترسی

سامانه شامل سه نقش اصلی است:

دانشجو : این نقش باید بتواند لیست تمرین های فعال و توضیحات هر تمرین را مشاهده کند. همچنین توانایی ارسال کد HTML، CSS و JavaScript را داشته و پس از آن بتواند پیش نمایش خروجی کد را مشاهده کند. علاوه بر آن با امکان مشاهده نتیجه تست ها روی کدهای ارسالی، مشاهده بازخورد استاد یا دستیار و مشاهده تاریخچه ارسال های خود را داشته باشد. همچنین دانشجو نباید بتواند تمرین ایجاد کند، تست ها را تغییر دهد یا ارسال های سایر دانشجویان را مشاهده کند.

استاد / دستیار آموزشی : این نقش باید بتواند تمرین جدید ایجاد کرده و تمرین های خود را در صورت نیاز ویرایش کند. همچنین باید امکاناتی چون: فعال یا غیرفعال کردن نمایش تمرین برای دانشجویان، تعریف تست های ارزیابی ساده برای هر تمرین، مشاهده کدهای ارسال شده توسط دانشجویان مشاهده نتیجه تست های هر دانشجو، ثبت بازخورد دستی روی پاسخ ها و تغییر وضعیت بررسی ارسال را دارا باشد.

مدیر سامانه (ادمین) : مدیر باید بتواند کاربران و نقش آن ها را مشاهده کند و در صورت لزوم نقش ها را تغییر دهد. همچنین این نقش باید بتواند تمامی تمرین ها و گزارش کلی ارسال ها را مشاهده کند و بصورت کلی به داده های سامانه دسترسی کامل داشته باشد. لازم به ذکر است که کنترل تمام دسترسی ها باید در سمت سرور و بر اساس این نقش ها انجام شود.

۳. سناریوی عملکرد سامانه

در ابتدا کاربر در سامانه ثبت نام می کند یا وارد حساب کاربری خود می شود. کاربری با نقش Instructor یا TA وارد پنل آموزشی شده و یک تمرین جدید تعریف می کند. برای هر تمرین، عنوان، توضیحات، مهلت ارسال، وضعیت انتشار (فعال/غیرفعال) و چند تست اعتبارسنجی تعیین می شود.

مثال تمرین: «یک فرم ورود طراحی کنید که شامل دو input برای نام کاربری و رمز عبور، یک دکمه ورود و یک div برای نمایش پیام باشد. با کلیک روی دکمه، متن div به مقدار مشخصی تغییر کند.» استاد یا دستیار آموزشی برای این تمرین چند تست تعریف می کند؛ مثلاً:

- وجود یک عنصر با id مشخص در DOM
- وجود حداقل دو تگ input
- وجود یک button با متن مشخص
- اعمال شدن یک class خاص روی فرم
- تغییر مقدار textContent یک div پس از کلیک روی دکمه

دانشجو پس از ورود، لیست تمرین های فعال را می بیند. یکی از تمرین ها را باز کرده و در سه بخش جداگانه، کدهای HTML، CSS و JS خود را وارد می کند. سپس در بخش Preview، خروجی نهایی کد را مشاهده می کند. پس از ارسال پاسخ، سامانه تست های تعریف شده را اجرا کرده و نتیجه را ذخیره و نمایش می دهد (شامل تعداد تست های موفق، ناموفق و پیام توضیحی هر تست).

استاد نیز با مراجعه به پنل خود، این ارسال ها را بررسی کرده و در صورت لزوم برای دانشجو بازخورد متنی ثبت می کند.

(نکته کلیدی: اجرای کد دانشجو باید فقط در محیط مرورگر و به صورت کنترل شده انجام شود و نیازی به اجرای کد کاربر در سمت سرور (Backend) نیست).

۴. فناوری ها و معماری

Frontend

- استفاده از HTML, CSS, JavaScript
- استفاده از DOM برای نمایش Preview (ترجیحاً درون iframe مجزا) و ارزیابی نتیجه تست ها
- استفاده از Event برای مدیریت دکمه ها و فرم های سامانه
- ارتباط با سرور صرفاً از طریق Fetch API

Backend

- محیط اجرا: Node.js
- فریم ورک: Express.js برای پیاده سازی RESTful API
- استفاده از Middleware برای احراز هویت، خواندن توکن ها و کنترل نقش ها
- ساختاردهی کد به صورت ماژولار (تفکیک Route ها، Controller ها و Model ها)

Database

- پایگاه داده: MongoDB
- استفاده از کالکشن ها برای ذخیره Users، Exercises، Tests، Submissions و Results

Authorization & Authentication

- پیاده‌سازی با JWT (همراه با تاریخ انقضا و امکان خروج/Logout)
- دریافت امن اطلاعات کاربر فعلی بر اساس توکن
- کنترل دسترسی مبتنی بر نقش (RBAC) در سطح مسیرهای سرور
- محدودسازی مشاهده ارسال‌ها صرفاً برای صاحب اثر یا مدرسین

۵. رابط‌های برنامه‌نویسی API (الزامی)

احراز هویت (Auth)

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/me

تمرین‌ها (Exercises)

- GET /api/exercises
- GET /api/exercises/:id
- POST /api/exercises

(مخصوص Admin و TA، Instructor)

- PATCH /api/exercises/:id

(مخصوص Admin و TA، Instructor)

- PATCH /api/exercises/:id/publish (فعال یا غیرفعال کردن تمرین)
- DELETE /api/exercises/:id

(مخصوص Admin یا ایجادکننده تمرین)

تست‌های تمرین (Tests)

- GET /api/exercises/:id/tests
- POST /api/exercises/:id/tests

(مخصوص Admin و TA، Instructor)

- PATCH /api/tests/:id

(مخصوص Admin و TA، Instructor)

- DELETE /api/tests/:id

(مخصوص Admin و TA، Instructor)

ارسال پاسخ (Submissions)

- POST /api/exercises/:id/submissions

(مخصوص Student)

- GET /api/submissions/me (ارسال‌های کاربر فعلی)
- GET /api/exercises/:id/submissions

(مخصوص Admin و TA، Instructor)

- GET /api/submissions/:id
- POST /api/submissions/:id/run-tests
- PATCH /api/submissions/:id/feedback

(مخصوص Admin و TA، Instructor)

گزارش‌ها (Reports)

- GET /api/reports/exercises
- GET /api/reports/submissions
- GET /api/reports/users

۶. رابط کاربری (UI)

حداقل صفحات مورد انتظار برای KNJudge عبارت‌اند از:

- صفحه ورود و ثبت‌نام
 - داشبورد دانشجو
 - صفحه لیست تمرین‌ها
 - صفحه جزئیات تمرین
 - صفحه حل تمرین (شامل سه بخش مجزا برای کدهای HTML، CSS و JavaScript)
 - بخش Preview خروجی کد دانشجو
 - بخش نمایش نتیجه ارزیابی تست‌ها (مثلاً با رنگ‌های سبز برای موفق و قرمز برای ناموفق)
 - صفحه تاریخچه ارسال‌های دانشجو
 - پنل استاد برای تعریف تمرین و تنظیمات آن
 - پنل تعریف تست‌ها برای تمرین
 - پنل بررسی ارسال‌ها و ثبت بازخورد توسط مدرس
 - پنل مدیریت کاربران برای Admin
- (در صفحه حل تمرین، محیط باید به گونه‌ای طراحی شود که دانشجو بتواند همزمان کدها و خروجی زنده آن را مشاهده کند).

۷. ملاحظات امنیتی

- کدهای دریافتی از کاربر (HTML/CSS/JS) به هیچ‌وجه نباید سمت سرور (Node.js) اجرا شوند.
- برای جلوگیری از تداخل استایل‌ها و اسکریپت‌ها، Preview حتماً در یک محیط ایزوله مانند iframe نمایش داده شود.
- دسترسی به سورس‌کد و نتایج ارسال‌های هر دانشجو برای سایر دانشجویان مسدود شود.
- نتایج تست‌ها پس از اجرا باید در دیتابیس ثبت شوند و صرفاً متکی به مقادیر Frontend نباشند.
- داده‌های ورودی API‌ها در سمت سرور Validate شوند.
- رمز عبور کاربران پیش از ذخیره در MongoDB با الگوریتمی مانند bcrypt هش شود.
- حجم کدهای ارسالی (Payload) محدود شود تا از اشغال بی‌مورد فضای سرور جلوگیری گردد.

۸. بخش‌های امتیازی (Bonus)

در صورت پیاده‌سازی موارد زیر، امتیاز اضافی در نظر گرفته می‌شود:

- ذخیره نسخه‌های متعدد (Revisions) از ارسال‌های دانشجو برای یک تمرین.
- نمایش درصد موفقیت تست‌ها (Coverage ساده).
- امکان اجرای مجدد تست توسط دانشجو قبل از ارسال نهایی.
- مقایسه بصری و نمایش تفاوت نتیجه ارسال‌های قبلی و جدید.
- پیاده‌سازی محدودیت زمانی (Timer) برای ارسال برخی تمرین‌ها.
- طراحی Leaderboard ساده بر اساس تعداد تست‌های موفق و سرعت حل.
- پیاده‌سازی محیط کدنویسی پیشرفته‌تر (شبیه به ویرایشگرهای واقعی با Syntax Highlighting).
- گزارش عملکرد و پیشرفت هر دانشجو در قالب نمودار.

۹. خروجی مورد انتظار

خروجی نهایی این پروژه باید شامل موارد زیر باشد:

- سورس کد کامل Frontend و Backend
- طراحی استاندارد دیتابیس MongoDB با داکيومنت‌های مشخص
- امکان پیاده‌سازی کامل سناریو: ورود استاد -> تعریف تمرین -> ورود دانشجو -> حل و مشاهده خروجی -> ثبت و اجرای تست -> بررسی توسط استاد
- قابل اتکا بودن سامانه و عدم وجود باگ در جریان اصلی (Main Flow)

دانشگاه صنعتی خواجه نصیرالدین طوسی

پروژه ۲: سامانه سفارش غذا و مدیریت صف آشپزخانه (FoodOps)

۱. معرفی سامانه

FoodOps یک سامانه تحت وب برای مدیریت سفارش غذا، نوشیدنی یا محصولات یک بوفه، کافه یا سلف کوچک است که پیاده سازی آن قرار است به دلیل تسلط کیومرث روی مبانی برنامه نویسی وب، به او سپرده شود. این سامانه به کاربران اجازه می‌دهد منو را مشاهده کرده، آیتم‌های مورد نظر خود را انتخاب نموده و سفارش خود را ثبت کنند. از طرف دیگر، کارکنان آشپزخانه می‌توانند سفارش‌ها را در صف کاری مشاهده کرده و وضعیت آن‌ها را به‌روزرسانی کنند. هدف اصلی این سامانه، مکانیزه کردن فرآیند ثبت سفارش، آماده‌سازی و تحویل است. به جای ثبت دستی، تمام مراحل از انتخاب آیتم تا تحویل نهایی در سامانه ذخیره می‌شود و کاربران می‌توانند وضعیت سفارش خود را پیگیری کنند. این پروژه باید یک جریان کاری کامل شامل: تعریف منو، ثبت سفارش، مدیریت صف آشپزخانه، تغییر وضعیت سفارش و تحویل نهایی را در بر گیرد.

۲. نقش‌ها و سطوح دسترسی

سامانه شامل چهار نقش اصلی است:

مشتری

- مشاهده منو و دسته‌بندی غذاها و نوشیدنی‌ها
- افزودن آیتم‌ها به سبد خرید و ثبت نهایی سفارش
- مشاهده سفارش‌های خود و وضعیت فعلی آن‌ها
- لغو سفارش (صرفاً در مراحل مجاز و پیش از شروع آماده‌سازی)

کارکنان آشپزخانه

- مشاهده صف سفارش‌های جدید بر اساس زمان ثبت
- مشاهده جزئیات هر سفارش
- تغییر وضعیت سفارش به «در حال آماده‌سازی»
- تغییر وضعیت سفارش به «آماده تحویل» پس از اتمام کار

صندوق‌دار / مسئول تحویل

- مشاهده لیست سفارش‌های آماده تحویل
- تطبیق کد یا شماره سفارش با مشتری
- ثبت تحویل نهایی و بستن سفارش

مدیر سامانه

- مدیریت کاربران و نقش‌های سیستمی
- مدیریت دسته‌بندی‌ها و آیتم‌های منو (ایجاد، ویرایش، حذف و تغییر موجودی)
- مشاهده گزارش‌های فروش و تعداد سفارش‌ها
- دسترسی کامل و نظارت بر کلیه سفارش‌ها

۳. سناریوی عملکرد سامانه

در ابتدا، مدیر سامانه وارد سامانه شده و دسته‌بندی‌های منو (مانند غذا، نوشیدنی، دسر) را تعریف می‌کند. سپس آیتم‌های زیرمجموعه هر دسته‌بندی را با اطلاعاتی نظیر نام، توضیحات، قیمت، موجودی، تصویر و وضعیت فعال/غیرفعال ثبت می‌کند. مشتری پس از ورود، منو را مشاهده کرده و آیتم‌های دلخواه را به سبد خرید اضافه می‌کند. سبد خرید باید منحصراً در سمت کاربر (توسط DOM و JavaScript) مدیریت شده و مبلغ نهایی پیش از تایید نهایی محاسبه و نمایش داده شود.

پس از ثبت نهایی، سفارش در دیتابیس ذخیره شده و وضعیت اولیه آن «ثبت‌شده» یا «در انتظار آماده‌سازی» خواهد بود. سپس پرسنل آشپزخانه از طریق پنل اختصاصی خود، این سفارش را در صف کاری مشاهده می‌کنند. آن‌ها وضعیت سفارش را ابتدا به «در حال آماده‌سازی» و پس از تکمیل، به «آماده تحویل» تغییر می‌دهند. در مرحله آخر، مسئول تحویل یا صندوق‌دار لیست غذاهای آماده را مشاهده کرده و پس از تحویل به مشتری، وضعیت را به «تحویل‌شده» تغییر می‌دهد. در تمامی این مراحل، مشتری می‌تواند وضعیت لحظه‌ای سفارش خود را در پنل کاربری‌اش مشاهده کند.

۴. فناوری‌ها و معماری

- **Frontend:** پیاده‌سازی با HTML، CSS و JavaScript. مدیریت سبد خرید با DOM و مدیریت رویدادها (Events) برای افزودن/حذف آیتم‌ها. ارتباط با سرور صرفاً از طریق Fetch API. متمایز کردن وضعیت سفارش‌ها با رنگ‌ها و برجسب‌های گرافیکی.
- **Backend:** استفاده از Node.js و Express.js برای توسعه RESTful API. استفاده از Middleware‌ها برای احراز هویت، کنترل دسترسی‌ها و مدیریت وضعیت سفارشات.
- **Database:** پایگاه داده MongoDB. طراحی کالکشن‌ها به صورت مجزا برای Users, Roles, MenuItems, Categories, Orders.
- **Authorization & Authentication:** استفاده از توکن JWT با تاریخ انقضا. پیاده‌سازی سیستم RBAC (محدودسازی مدیریت منو به Admin، مدیریت صف به کارکنان آشپزخانه و تحویل به صندوقدار). جلوگیری از دسترسی هر کاربر به سفارش‌های سایر مشتریان.

۵. رابط‌های برنامه‌نویسی API (الزامی)

احراز هویت

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/me

منو و دسته‌بندی‌ها

- GET /api/categories
- POST /api/categories

(مخصوص Admin)

- PATCH /api/categories/:id

(مخصوص Admin)

- DELETE /api/categories/:id

(مخصوص Admin)

- GET /api/menu-items
- GET /api/menu-items/:id
- POST /api/menu-items

(مخصوص Admin)

- PATCH /api/menu-items/:id

(مخصوص Admin)

- PATCH /api/menu-items/:id/availability

(مخصوص Admin)

- DELETE /api/menu-items/:id

(مخصوص Admin)

سفارش‌ها

- POST /api/orders

(مخصوص Customer)

- GET /api/orders/me (سفارش‌های کاربر فعلی)
- GET /api/orders/:id
- PATCH /api/orders/:id/cancel

(مخصوص Customer، در مرحله مجاز)

صف آشپزخانه

- GET /api/kitchen/orders

(مخصوص Admin و Kitchen Staff)

- PATCH /api/orders/:id/start

(تغییر وضعیت به در حال آماده‌سازی)

- PATCH /api/orders/:id/ready

(تغییر وضعیت به آماده تحویل)

تحویل سفارش

- GET /api/delivery/orders (سفارش‌های آماده تحویل)
- PATCH /api/orders/:id/deliver (ثبت تحویل سفارش)

مدیریت و گزارش

- GET /api/admin/orders
- GET /api/admin/reports/daily
- GET /api/admin/reports/items
- PATCH /api/admin/users/:id/role

۶. رابط کاربری (UI)

حداقل صفحات مورد انتظار برای FoodOps عبارت‌اند از:

- صفحه ورود و ثبت نام
 - صفحه نمایش منو (آیتم‌ها باید در قالب کارت شامل نام، توضیح، قیمت، وضعیت موجودی و دکمه افزودن نمایش داده شوند)
 - صفحه جزئیات آیتم
 - بخش مدیریت سبد خرید
 - صفحه نهایی کردن و ثبت سفارش
 - صفحه سفارش‌های من (پیگیری وضعیت)
 - پنل آشپزخانه برای مشاهده صف سفارش‌ها (مرتب‌شده بر اساس زمان)
 - پنل تحویل سفارش برای صندوق‌دار
 - پنل مدیریت منو و دسته‌بندی‌ها برای ادمین
 - پنل گزارش‌های مدیریتی
- (وضعیت‌های سفارش در پنل‌ها باید کاملاً واضح و با لیبل‌های رنگی مجزا مانند: ثبت‌شده، در حال آماده‌سازی، آماده تحویل، تحویل‌شده و لغوشده نمایش داده شوند).

۷. ملاحظات امنیتی

- رمز عبور کاربران پیش از ذخیره در پایگاه داده باید با الگوریتم bcrypt هش شود.
- **بسیار مهم:** قیمت نهایی سفارش به هیچ‌وجه نباید متکی به داده‌های ارسالی از Frontend (مثل سبد خرید) باشد؛ سرور موظف است قیمت تک‌تک آیتم‌ها را از دیتابیس خوانده و جمع کل را خودش محاسبه کند.
- کاربر تحت هیچ شرایطی نباید بتواند سفارش افراد دیگر را مشاهده یا لغو کند.
- تغییر وضعیت سفارش‌ها صرفاً توسط نقش‌های مجاز و طبق منطق جریان کاری مجاز است (مثلاً یک سفارش "تحویل‌شده" نباید بتواند به "در حال آماده‌سازی" برگردد).
- موجودی آیتم‌ها باید حتماً در سمت سرور بررسی شود.
- داده‌های ورودی اعتبارسنجی شوند و برای جلوگیری از حملات NoSQL Injection پاک‌سازی گردند.
- عملیات حساس سیستم (مانند لغو یا تحویل نهایی) بهتر است در یک کالکشن مجزا برای لاگ‌گیری امنیتی ثبت شوند.

۸. بخش‌های امتیازی (Bonus)

در صورت پیاده‌سازی موارد زیر، امتیاز اضافی به گروه تعلق می‌گیرد:

- مدیریت خودکار موجودی آیتم‌ها (کاهش موجودی پس از ثبت هر سفارش).
- جلوگیری از امکان افزودن آیتم ناموجود به سبد خرید.
- اعمال محدودیت زمانی برای سفارش‌گیری (تعریف ساعات کاری).
- نمایش سفارش‌های آماده تحویل روی یک صفحه عمومی (مناسب برای نمایشگر سلف).
- امکان فیلتر کردن سفارش‌ها بر اساس تاریخ و وضعیت.
- قابلیت جستجو در بین غذاهای منو.

- گزارش آماری از پرفروش‌ترین آیتم‌ها و حجم سفارش‌های روزانه.
- قابلیت تعریف و اعمال کد تخفیف در سیستم.
- ثبت لاگ دقیق تاریخچه تغییر وضعیت هر سفارش شامل: `new_status`, `old_status`, `order_id`, `changed_at`, `changed_by`.
- نمایش زمان تقریبی برای آماده‌سازی سفارش به مشتری.
- طراحی داشبورد اختصاصی و منسجم برای پرسنل.

۹. خروجی مورد انتظار

خروجی نهایی این پروژه باید شامل موارد زیر باشد:

- پیاده‌سازی کامل سورس‌کد Frontend و Backend
- طراحی استاندارد دیتابیس MongoDB با تعریف صحیح کالکشن‌ها
- اجرای یک جریان کاری (Flow) بی‌نقص؛ از تعریف منو توسط ادمین تا ثبت سبد خرید، مدیریت در آشپزخانه و تحویل به مشتری
- رابط کاربری کارآمد و API کاملاً عملیاتی

دانشگاه صنعتی خواجه نصیرالدین طوسی

پروژه ۳: سامانه ساخت و تست Mock API (MockLab)

۱. معرفی سامانه

پس از آنکه کیومرث دو پروژه قبلی را از آن خود کرد، حالا نوبت فریبرز است تا خودی نشان دهد. او با ارائه پروپوزال MockLab، که یک سامانه تحت وب برای ساخت و شبیه‌سازی API‌های آزمایشی (Mock APIs) است، قصد دارد کمک شایانی به برنامه نویسان ایران در جهت استقلال از پلتفرم‌های خارجی کند. در این سامانه، کاربران توسعه‌دهنده می‌توانند برای پروژه‌های خود Endpoint‌های فرضی تعریف کنند. هر Endpoint شامل مسیر (Path)، متد HTTP، کد وضعیت پاسخ، بدنه پاسخ به فرمت JSON یا متن و سایر تنظیمات اولیه است. پس از تعریف، هر زمان که این مسیر اختصاصی فراخوانی شود، سامانه پاسخ تعریف‌شده را عموماً برمی‌گرداند. هدف اصلی این سامانه، کمک به توسعه‌دهندگان برای تست بخش Frontend یا شبیه‌سازی منطق Backend پیش از پیاده‌سازی واقعی آن است. این پروژه ارتباط مستقیمی با مفاهیم پایه‌ای درس مهندسی اینترنت دارد؛ از جمله: HTTP، URL، Route، Request، Response، Header، Query String، Body، Status Code و معماری RESTful API. (تذکر: این سامانه صرفاً داده‌های ثابت متنی یا JSON را ذخیره و باز می‌گرداند و به هیچ‌وجه شامل اجرای کدهای ارسالی یا دلخواه کاربر روی سرور نخواهد بود).

۲. نقش‌ها و سطوح دسترسی

سامانه شامل سه نقش اصلی است:

توسعه‌دهنده

- ایجاد پروژه‌های Mock جدید
- تعریف، ویرایش و مدیریت Endpoint‌ها در پروژه‌های خود (تعیین متد، مسیر، کد وضعیت و بدنه پاسخ)
- فعال یا غیرفعال کردن موقت Endpoint‌ها
- تست زنده Endpoint‌ها از داخل پنل کاربری
- مشاهده و بررسی لاگ درخواست‌های ارسالی به Endpoint‌های متعلق به خود

مدیر پروژه

- مشاهده پروژه‌های Mock مربوط به تیم یا گروه خود
- مدیریت وضعیت فعال/غیرفعال بودن Endpoint‌های پروژه
- مشاهده لاگ‌ها، گزارش‌های استفاده و آمارها
- مدیریت اعضای پروژه در سطح دسترسی‌های ساده

مدیر سامانه

- مدیریت کاربران و تغییر نقش‌های سیستمی
- مشاهده و نظارت بر تمامی پروژه‌های Mock ایجادشده در سامانه
- مشاهده گزارش جامع میزان استفاده از منابع سرور
- غیرفعال کردن پروژه‌ها یا Endpoint‌های نامعتبر، مخرب یا حجیم

۳. سناریوی عملکرد سامانه

کاربر توسعه دهنده پس از ورود به سامانه، یک پروژۀ Mock (مثلاً به نام "Shop API") ایجاد می‌کند. سپس در این پروژۀ Endpoint های مورد نیاز خود را تعریف می‌کند؛ برای نمونه:

- **Method :** GET
- **Path :** /products
- **Status Code :** 200
- **Response Body :** json
[
 { "id": 1, "title": "Keyboard", "price": 1200 },
 { "id": 2, "title": "Mouse", "price": 700 }
]

سامانه برای این پروژۀ یک مسیر پایه و منحصر به فرد ایجاد می‌کند (مانند mock/shop-api/products/). هرگاه این آدرس توسط مرورگر، Fetch API یا ابزارهای تست API فراخوانی شود، سامانه همان پاسخ JSON تعریف شده را همراه با وضعیت مشخص شده (200 OK) برمی‌گرداند. کاربر می‌تواند متدهای مختلف نظیر GET, POST, PATCH, DELETE را شبیه‌سازی کند. به ازای هر درخواستی که به این آدرس‌های فرضی ارسال می‌شود، سامانه اطلاعاتی شامل زمان دقیق، متد، مسیر، Query String، بدنه درخواست و کد وضعیت پاسخ را در قالب لاگ ذخیره می‌کند تا توسعه‌دهنده بتواند در پل خود رفتار کلاینت‌ها را رصد کند.

۴. فناوری‌ها و معماری

- **Frontend:** پیاده‌سازی فرم‌های پویا برای ساخت پروژۀ و Endpoint، اعتبارسنجی اولیه ساختار صحیح JSON در مرورگر، نمایش لاگ‌ها در قالب جدول و استفاده از Fetch API برای تست و شبیه‌سازی درخواست‌ها.
- **Backend:** استفاده از Node.js و Express.js. استفاده از فرآیند مسیریابی پویا (Dynamic Routing) برای تشخیص و پاسخ‌دهی به آدرس‌های داینامیک Mock. پیاده‌سازی Middleware ها جهت احراز هویت، ثبت لاگ درخواست‌ها و بررسی دسترسی‌ها.
- **Database:** پایگاه داده MongoDB. ذخیره اطلاعات در کالکشن‌های مجزا برای Users, Projects, Endpoints, ResponseSettings, RequestLogs. بدنه پاسخ‌ها (Response Body) باید به صورت متن یا ساختار BSON/JSON ذخیره گردد.
- **Authorization & Authentication:** سیستم احراز هویت با JWT (همراه تاریخ انقضا و دکه خروج). پیاده‌سازی کنترل دسترسی مبتنی بر نقش (RBAC) و محدودسازی دسترسی به پروژۀها و لاگ‌ها صرفاً برای مالکین یا ادمین.

۵. رابط‌های برنامه‌نویسی API (الزامی)

احراز هویت

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/me

پروژه‌های Mock

- GET /api/mock-projects
- POST /api/mock-projects
- GET /api/mock-projects/:id
- PATCH /api/mock-projects/:id
- DELETE /api/mock-projects/:id
- PATCH /api/mock-projects/:id/activate
- PATCH /api/mock-projects/:id/deactivate

مدیریت Endpointها

- GET /api/mock-projects/:id/endpoints
- POST /api/mock-projects/:id/endpoints
- GET /api/endpoints/:id
- PATCH /api/endpoints/:id
- DELETE /api/endpoints/:id
- PATCH /api/endpoints/:id/activate
- PATCH /api/endpoints/:id/deactivate

تست Endpoint

- POST /api/endpoints/:id/test
- GET /api/endpoints/:id/example-url

لاگ درخواست‌ها

- GET /api/endpoints/:id/logs
- GET /api/mock-projects/:id/logs
- DELETE /api/endpoints/:id/logs

مسیر عمومی و پویای Mock (هسته سامانه)

- GET /mock/:projectKey/*
- POST /mock/:projectKey/*
- PATCH /mock/:projectKey/*
- DELETE /mock/:projectKey/*

(در این مسیرهای عمومی، سرور باید بر اساس projectKey، بقیه آدرس فرستاده‌شده و متد HTTP، سند مربوطه را در دیتابیس جستجو کرده و پاسخ تعریف‌شده کاربر را بازگرداند).

- GET /api/admin/mock-projects
- GET /api/admin/reports/usage
- PATCH /api/admin/users/:id/role

۶. رابط کاربری (UI)

حداقل صفحات مورد انتظار برای MockLab عبارت‌اند از:

- صفحه ورود و ثبت‌نام
- داشبرد پروژه‌های کاربری
- فرم ایجاد پروژه جدید و فرم‌های تفکیک‌شده ساخت و ویرایش Endpoint
- صفحه جزئیات پروژه و لیست تمام مسیرهای تعریف‌شده برای آن
- صفحه اختصاصی تست آنلاین Endpointها
- صفحه نمایش جداول لاگ (شامل ستون‌های: زمان، متد، مسیر، query، کد وضعیت، IP یا شناسه کلاینت و خلاصه‌ای از Body)
- پنل گزارش میزان استفاده و پنل مدیریت کاربران (مخصوص Admin)

۷. ملاحظات امنیتی

- رمز عبور کاربران با کتابخانه bcrypt هش شود.
- تایید هویت و مالکیت پروژه در تمامی Endpointهای حساس چک شود تا توسعه‌دهنده‌ای نتواند مسیرهای دیگران را تغییر داده یا لاگ‌های آن‌ها را ببیند.
- **بسیار مهم:** محتوای متنی بدنه پاسخ به هیچ عنوان نباید به عنوان کد اجرایی در سمت سرور Evaluate یا تایید شود.
- مسیرهای Mock تعریف‌شده توسط کاربر باید اعتبارسنجی شوند؛ سامانه نباید اجازه دهد کاربران مسیرهای داخلی و حیاتی سرور (مانند /api/auth/login) را بازنویسی یا مختل کنند.
- حجم پاسخ‌ها و همچنین مقدار تأخیر اعمالی باید سقف مشخصی داشته باشند تا جلوی حملات دیسک یا کندی عمده سرور گرفته شود.
- برای جلوگیری از حملات NoSQL Injection، تمامی ورودی‌های ارسالی به دیتابیس پاک‌سازی شوند.
- از تعریف Routeهای کاملاً تکراری (متد و مسیر یکسان) در یک پروژه واحد جلوگیری به عمل آید.

۸. بخش‌های امتیازی (Bonus)

در صورت پیاده‌سازی موارد زیر، امتیاز اضافی لحاظ خواهد شد:

- امکان تعریف پارامتر تأخیر اختیاری (Delay) بر حسب میلی‌ثانیه برای شبیه‌سازی سرعت پایین اینترنت در پاسخ API.
- قابلیت تعریف چند پاسخ (Response) مختلف برای یک Endpoint واحد و انتخاب تصادفی یا شرطی یکی از آن‌ها در زمان فراخوانی.
- پیاده‌سازی فیلترینگ پیشرفته روی جدول لاگ‌ها (بر اساس تاریخ، متد و Status Code).
- نمایش نمودارها و آمارهای گرافیکی از تعداد درخواست‌های روزانه.

- قابلیت خروجی گرفتن (Export) و وارد کردن (Import) کل تنظیمات یک پروژه به صورت فایل JSON.
- امکان ایجاد Token اختصاصی برای هر پروژه جهت محدود کردن دسترسی و امن سازی فراخوانی آدرس‌های Mock.
- نمایش نمونه کد کپی‌شدنی Fetch API جاوااسکریپت برای هر Endpoint جهت کپی راحت در فرانت‌اند دانشجویان.
- طراحی و ثبت کالکشن audit_logs برای کارهای مدیریتی و حساس شامل فیلدهای: actor_id, action, entity_id, entity_type, created_at.
- امکان فعال یا غیرفعال سازی یک‌جای کل پروژه.

۹. خروجی مورد انتظار

- سورس کد کامل پروژه در دو بخش Frontend و Backend.
- ساختار کالکشن‌های دیتابیس در MongoDB.
- امکان تست کامل فرآیند: ساخت پروژه -> تعریف Endpoint -> فراخوانی آدرس فرضی -> دریافت پاسخ استاتیک -> ثبت و نمایش لاگ در پنل.
- کد تمیز و API‌های کاملاً استاندارد و مستندشده.

جمع‌بندی پروژه‌ها

سه پروژه معرفی شده در این سند عبارت‌اند از:

1. **KNJudge**: سامانه تمرین و ارزیابی خودکار کدهای HTML/CSS/JS

2. **FoodOps**: سامانه سفارش غذا و مدیریت صف آشپزخانه

3. **MockLab**: سامانه ساخت، شبیه‌سازی و تست API Mock

هر سه پروژه در سطح استانداردهای عملی طراحی شده‌اند و باید به صورت یک سامانه کامل تحت وب پیاده‌سازی شوند. وجود بخش‌های Backend (با Node.js/Express)، Frontend، پایگاه داده (MongoDB)، سیستم احراز هویت (JWT)، کنترل دسترسی مبتنی بر نقش (RBAC)، طراحی کامل RESTful API و رابط کاربری کارآمد در همگی آن‌ها الزامی است.

انتظار می‌رود گروه‌ها پروژه انتخابی خود را به گونه‌ای پیاده‌سازی کنند که در زمان تحویل، سناریوی کاری اصلی آن بدون هیچ باگ یا خطایی قابل ارائه و نمایش باشد. خروجی نهایی این پروژه‌ها، ابزار اصلی ارزیابی میزان آشنایی و تسلط دانشجویان بر مفاهیم کلیدی و عملی درس مهندسی اینترنت، شامل تعامل فرانت و بک‌اند، دستکاری DOM، کار با دیتابیس‌های مدرن و استانداردهای امنیتی وب خواهد بود.

با آرزوی موفقیت برای تمامی دانشجویان